BOOKNEST

ONLINE BOOKSTORE

PROJECT REPORT

MERN Stack Web Application

Submitted by: Amoz Paul

Institution: sahrdaya college of engineering and technology thrissur

Course / Programme: B Tech CSE

Project guide: Dr.Anil Antony

Submission date: 24/10/2026

Repository: https://github.com/amoz225713-web/online-book-store

Host Link : https://amoz225713-web.github.io/online-book-store/

CONTENTS

1. EXECUTIVE SUMMARY
2. INTRODUCTION
3. OBJECTIVES
4. SCOPE AND DELIVERABLES
5. METHODOLOGY
6. PROJECT ACTIVITIES
7. SYSTEM DESIGN AND IMPLEMENTATION
8. RESULTS & FINDINGS
9. CONCLUSION
10. APPENDIX

EXECUTIVE SUMMARY

BookNest is a web-based online bookstore developed as a MERN-stack student project. It combines a React frontend, an Express and Node.js backend API, and a MongoDB database accessed through Mongoose. The frontend is deployed using GitHub Pages, the backend is hosted on Render, and book records are stored in MongoDB Atlas.

The application provides a foundation for browsing book information and searching by title, author, or category. The API supports listing, retrieving, creating, updating, and deleting book records. During deployment testing, the API initially returned an empty array while records existed in Atlas. The database path in the MongoDB connection string was corrected to select the intended bookstore database, after which the API returned book details.

INTRODUCTION

Online bookstores let readers explore book catalogues through a web interface. Such applications need a frontend for presentation, a backend for processing requests, and a database for persistent data.

BookNest was developed to practise full-stack web development using the MERN stack. React provides the browser interface; Node.js runs the server; Express defines HTTP routes; MongoDB stores book documents; and Mongoose defines the schema and performs database operations.

The browser communicates with the backend API, and the backend communicates with MongoDB. Database credentials should remain in environment variables and must not be committed to the public repository.

OBJECTIVES

11. 1. Build a bookstore interface using React and Vite.
12. 2. Create a REST-style API using Node.js and Express.
13. 3. Store and retrieve book records using MongoDB Atlas and Mongoose.
14. 4. Support book listing, search, single-book retrieval, creation, updating, and deletion through API endpoints.
15. 5. Use GitHub for version control and deploy the frontend and backend to hosting services.
16. 6. Practise troubleshooting using API responses, browser developer tools, deployment logs, CORS settings, and database configuration.

SCOPE AND DELIVERABLES

Included in the current implementation:

- React/Vite frontend for the bookstore.
- Express API with a health endpoint and book routes.
- Mongoose Book model with title, author, price, description, image, category, and stock fields.
- Optional search across title, author, and category.
- CRUD endpoints for book records.
- MongoDB Atlas database integration.
- GitHub repository and GitHub Pages frontend deployment.
- Render-hosted backend service.

Features not confirmed as implemented and therefore not claimed as completed: customer authentication, payment processing, shopping-cart checkout, order management, and role-based administration. These are possible future enhancements.

METHODOLOGY

The project was developed incrementally. The React client was configured with Vite, and an Express server was created with CORS, JSON parsing, routes, and error handling. A Mongoose schema was defined for book records.

The backend was connected to MongoDB Atlas through the MONGODB_URI environment variable. The API was deployed to Render. The frontend was configured to call the deployed API and was built for GitHub Pages. GitHub Actions was used to automate frontend deployment.

Testing began with the health endpoint and then the books endpoint. When the books endpoint returned an empty array despite records existing in Atlas, the database path in the connection string was checked and corrected to select the bookstore database.

PROJECT ACTIVITIES

17. 1. Project setup: organised source code into client and server areas and used Node.js/npm to manage dependencies.
18. 2. Frontend: used React and Vite to build the bookstore interface and request data from the API.
19. 3. Backend: implemented Express routes, CORS configuration, JSON parsing, and error handling.
20. 4. Data modelling: created a Mongoose schema with required title, author, and numeric price fields. Description and image default to empty strings; category defaults to General; stock defaults to 10.
21. 5. Search and CRUD: implemented list/search, retrieve-by-ID, create, update, and delete operations.
22. 6. Deployment: deployed the frontend to GitHub Pages and backend to Render, with MongoDB Atlas as the database.
23. 7. Troubleshooting: verified endpoints, checked the deployment workflow, and corrected the database path so the backend reads the intended collection.

SYSTEM DESIGN AND IMPLEMENTATION

Architecture: Browser (React/Vite) → HTTPS request → Express API on Render → Mongoose → MongoDB Atlas. The browser does not connect directly to MongoDB.

Data model: Book documents contain title (String, required), author (String, required), price (Number, required, minimum 0), description (String, default empty), image (String, default empty), category (String, default General), and stock (Number, default 10, minimum 0). Mongoose timestamps generate createdAt and updatedAt.

API endpoints:

- GET /api/health — returns a JSON health message.
- GET /api/books — returns books and accepts an optional search query.
- GET /api/books/:id — returns one book or a 404 response.
- POST /api/books — creates a book from a JSON request body.
- PUT /api/books/:id — updates a book and validates updated fields.

- DELETE /api/books/:id — deletes a book.

Error handling returns 400 for validation and invalid-ID errors, 404 when a requested book is not found, and 500 for other server errors.

RESULTS & FINDINGS

The deployed health endpoint responded successfully. The books endpoint initially returned an empty array while the Atlas interface showed book records. The database selection in the connection string was corrected, after which the API returned book details.

The GitHub Actions workflow completed successfully. This confirmed that the frontend deployment pipeline ran successfully; the live page still needs to be retested after later changes.

Testing summary: health endpoint checked; books endpoint returned book details after the database correction; records were visible in Atlas; frontend deployment workflow succeeded. Credentials should remain private, and any database password shared during troubleshooting should be rotated.

CONCLUSION

BookNest demonstrates the principal parts of a MERN-stack web application: a React client, an Express/Node.js API, a MongoDB database, source control, and cloud deployment. The project provided practical experience connecting frontend requests to backend routes and connecting Mongoose models to a managed database.

The troubleshooting process demonstrated that a successful deployment does not guarantee that the application is connected to the intended database. Future work can add automated tests, stronger validation, authentication and authorization for write operations, pagination, accessibility improvements, and a complete cart/order workflow.

APPENDIX

GitHub repository: https://github.com/amoz225713-web/online-book-store

Frontend: https://amoz225713-web.github.io/online-book-store/

Backend API base: https://booknest-api-xr87.onrender.com

Example checks: /api/health and /api/books

Official references: React (https://react.dev/), Node.js (https://nodejs.org/en/docs), Express (https://expressjs.com/), MongoDB (https://www.mongodb.com/docs/), MongoDB Atlas (https://www.mongodb.com/docs/atlas/), Mongoose (https://mongoosejs.com/docs/), GitHub Docs (https://docs.github.com/), and MDN CORS (https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS).